Big-O: Because Simple Words Were Too Easy

Yayati Gupta
February 9, 2026 (12:35–13:30)

Contents

1 Recap **1**

2 Today **1**
2.1 Key Intuitions from Complexity Analysis . . . 1
2.2 It's the Long-Term Growth that Matters . . . 1
2.3 Why Do We Need Big-O Notation If the Idea Is So Simple? . . . 2
2.4 Common Mistakes with Asymptotic Notation . . . 4
2.4.1 The Exponential Confusion . . . 4
2.4.2 Constant Confusion . . . 4
2.4.3 The Equality Mistake . . . 5
2.4.4 Big-O Is Not a Number . . . 5
2.4.5 Be Careful with Operations . . . 5

3 Coming next **6**
3.1 Guaranteed exam marks . . . 6

1 Recap

1. Understanding Recursive Binary Search
2. Building Intuition About Function Growth Rates

2 Today

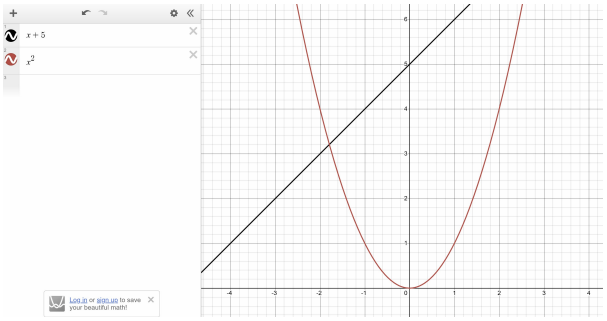
2.1 Key Intuitions from Complexity Analysis

1. **Constant additions and subtractions do not affect growth rate.**
   Example: $T(n) = n^2 + 1000$ behaves like n^2 for large n.
2. **Constant multiplications and divisions do not affect growth rate.**
   Examples: $T(n) = 5n^2$ and $T(n) = \frac{n^2}{10}$ both grow like n^2.
3. **When multiple terms are added, the highest-order term determines overall growth.**
   Example: $T(n) = n^3 + n^2 + n$ grows like n^3.
4. **Multiplying growth functions creates a different growth rate.**
   Example: $n \times \log n = n \log n$, which grows faster than n but slower than n^2.

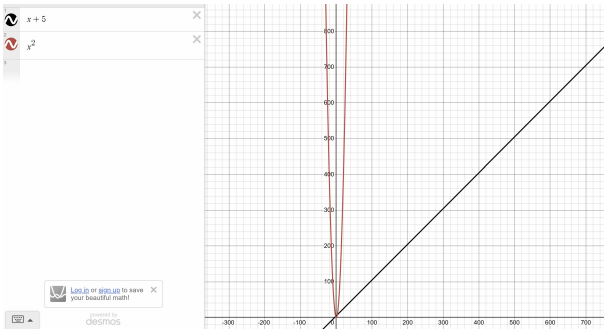
2.2 It's the Long-Term Growth that Matters

At small input sizes, a function with a constant addition may appear larger. For example, $f(x) = x+5$ can be greater than $g(x) = x^2$ when x is small. However, as x becomes large, x^2 grows much faster than $x + 5$. After some point, x^2 stays above $x + 5$ and the gap keeps increasing.

Key Idea: In complexity analysis, we are interested in long-term growth. We focus on which function grows faster as the input becomes very large, not who is bigger at the beginning.



Zoomed-in view: $x + 5$ appears larger than x^2



Zoomed-out view: x^2 eventually exceeds $x + 5$ and keeps growing faster

Tug of War: Polynomial vs Exponential Growth

At small values of x, it may seem like x^2 and 2^x take turns being larger. Sometimes the polynomial looks bigger, and sometimes the exponential shoots ahead.

Question 1: As x becomes very large, which function eventually grows faster?

$$x^2 \quad \text{or} \quad 2^x \, ?$$

Do you think one of them will stay larger forever after some point?

Question 2: Let's experiment with constants!
Try to find a constant c (start with $c = 1, 2, 3, 4, \ldots$) such that

$$2^x \geq c \cdot x^2$$

for all sufficiently large x.
You do not need the perfect value — just try different constants and see what happens as x increases.

What should you notice? No matter how large a constant multiple of x^2 you take, the exponential function 2^x eventually overtakes it and keeps growing faster.

2.3 Why Do We Need Big-O Notation If the Idea Is So Simple?

At an intuitive level, analyzing an algorithm's growth seems straightforward: identify the highest-order term in the running time expression. For example, in

$$f(n) = 2n + 3,$$

the term n dominates for large n, so we say the algorithm has linear growth.

This raises a natural question:

If we already know the dominant term, why do we need Big-O, Big-Ω, and Big-Θ notation at all?

Growth Shape vs Exact Value Asymptotic notation is not concerned with exact values of functions, but with their **shape of growth** as $n \to \infty$. Constants and lower-order terms affect the height of the curve, but not its overall growth behavior.

For example, consider $f(n) = 2n + 3$ and $g(n) = n$. It is not true that

$$2n + 3 \leq n$$

for large n. In fact, $2n + 3$ is always larger than n. So we cannot directly say $f(n) \leq g(n)$.

However, we can say that

$$2n + 3 \leq 3n \quad \text{for all } n \geq 3.$$

Here, $3n$ is just a constant multiple of n. This shows that $f(n)$ does not grow faster than a constant times n.

Why Multiply by a Constant? Big-O notation allows multiplication by a constant because constants do not change the growth type. They only stretch or shrink the graph vertically.

Without allowing constants, most real functions would fail to fit cleanly into any category:

- $1000n$ is not $\leq n$, but it is still linear.
- $0.5n^2$ is not $\leq n^2$, but it is still quadratic.

Thus, in Big-O we formally say:

$$f(n) \in O(g(n)) \quad \text{if there exist constants } c > 0 \text{ and } n_0$$

such that

$$f(n) \leq c\, g(n) \quad \text{for all } n \geq n_0.$$

This means that beyond some point, $f(n)$ is always bounded above by a scaled version of $g(n)$.

Big-Ω and Big-Θ Similarly,

- $f(n) \in \Omega(g(n))$ means $f(n)$ eventually grows at least as fast as $g(n)$ (up to constant factors).
- $f(n) \in \Theta(g(n))$ means $f(n)$ grows at the same rate as $g(n)$, being bounded both above and below by constant multiples of $g(n)$.

The Big Picture In summary, asymptotic notation does not complicate the idea of growth — it formalizes it.

Intuitively, we look at the highest-order term. Formally, we prove that after some point, the function stays within constant multiples of that dominant term.

Asymptotic notation is simply a precise language for saying: "In the long run, which function grows faster?"

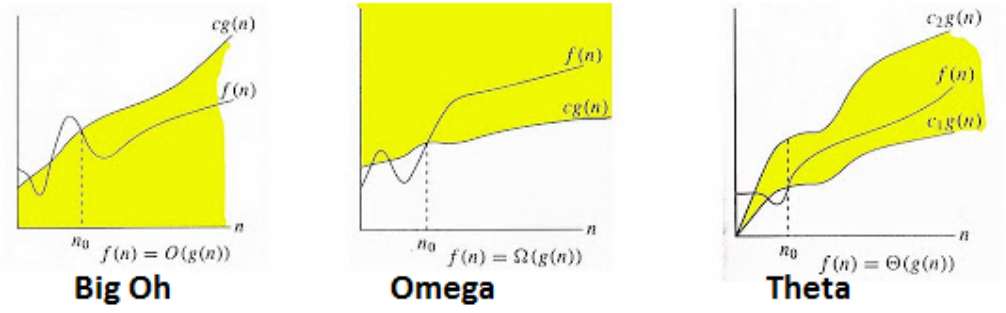


Figure 1: Source: Medium

Focus on Growth, Not the Scary Symbols!

Textbook graphs of O, Ω, and Θ may look confusing at first. But remember: those pictures and symbols are only there to make things mathematically precise.

If you have a function like

$$f(n) = 6n^2 + 9n + 2,$$

its growth **shape** is determined by the highest-order term, which is n^2. So this function grows **quadratically**.

Because of this, we can always find:

- A constant c_1 such that $f(n) \leq c_1 n^2$ for all large n
- A constant c_2 such that $f(n) \geq c_2 n^2$ for all large n

This simply means $f(n)$ grows at the **same rate** as n^2 in the long run.

What you really need to do is simple:

1. Identify the **highest-order term** $\longrightarrow$ this tells you the growth shape.
2. Choose the notation based on the type of analysis:
   - Best case $\longrightarrow \Omega(\cdot)$
   - Worst case $\longrightarrow O(\cdot)$
   - Tight/exact growth $\longrightarrow \Theta(\cdot)$

The shape comes first — the symbol comes after.

2.4 Common Mistakes with Asymptotic Notation

Asymptotic notation is powerful, but it is also very easy to misuse. Here are some common ways students (and sometimes textbooks!) get into trouble.

2.4.1 The Exponential Confusion

At first glance, someone might think:

$$4^x = O(2^x)$$

because 4 is just a constant multiple of 2.

But this is completely wrong.

$$4^x = (2^2)^x = 2^{2x} = (2^x)^2$$

So 4^x grows like the **square** of 2^x. That means it grows much, much faster.

With exponentials, small changes in the base can create huge differences in growth.

2.4.2 Constant Confusion

It is true that every constant is $O(1)$. For example,

$$17 = O(1)$$

But this fact can be misused.

Look at this wrong idea:

$$\sum_{i=1}^{n} i = 1 + 2 + 3 + \cdots + n$$

Someone might say: each i is a constant, so

$$i = O(1)$$

and therefore

$$\sum_{i=1}^{n} i = O(1) + O(1) + \cdots + O(1) = O(n)$$

This is wrong.

Why? Because i is not a fixed constant — it changes and grows with n. Also, we should not treat $O(1)$ like a number and just add it repeatedly.

In reality,

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2} = \Theta(n^2)$$

2.4.3 The Equality Mistake

We write

$$f(n) = O(g(n))$$

but the "=" does not mean normal equality.

It only means that $f(n)$ grows no faster than a constant multiple of $g(n)$ for large n.

If we treat "=" like real equality, we can get nonsense. For example:

$$2n = O(n) \quad \text{and} \quad n = O(n)$$

If we pretend these are equalities, we might wrongly conclude:

$$n = 2n$$

Clearly impossible!

So remember: Big-O statements describe a **growth relationship**, not an equation you can rearrange.

2.4.4 Big-O Is Not a Number

Expressions like $O(n)$ or $O(1)$ are not actual values. They only make sense in statements like

$$f(n) = O(g(n))$$

So writing things like

$$O(1) + O(1) = O(2)$$

as if they are normal numbers is not correct.

2.4.5 Be Careful with Operations

Not all math operations keep asymptotic relationships safe.

Just because two functions grow at similar rates does not mean applying powers or exponentials to both will keep them similar.

However, some operations do behave nicely. For example, if

$$f(n) = \Theta(g(n)),$$

then

$$\log f(n) = \Theta(\log g(n))$$

But in general, be careful when squaring, exponentiating, or transforming asymptotic expressions.

Main Message: Asymptotic notation is about long-term growth, not normal algebra. Always think about how the functions grow as n becomes large.

Same Function, Multiple Asymptotic Relations

A function can belong to many Big-O classes!
Consider the function $f(n) = n$.

Big-O (Upper Bound):
We say $f(n) = O(g(n))$ if $f(n)$ does not grow faster than $g(n)$ (up to constant factors).
Since n grows slower than n^2, n^3, and $n \log n$, we can write:

$$n \in O(n), \quad n \in O(n \log n), \quad n \in O(n^2), \quad n \in O(2^n)$$

All of these are true because each function on the right eventually grows at least as fast as n.

Big-Ω (Lower Bound):
We say $f(n) = \Omega(g(n))$ if $f(n)$ does not grow slower than $g(n)$.
Since n grows faster than $\log n$ and constants, we have:

$$n \in \Omega(1), \quad n \in \Omega(\log n), \quad n \in \Omega(n)$$

But $n \notin \Omega(n^2)$ because n eventually becomes much smaller than n^2.

Big-Θ (Tight Bound):
We say $f(n) = \Theta(g(n))$ only when both $O(g(n))$ and $\Omega(g(n))$ are true.
For $f(n) = n$:

$$n \in \Theta(n)$$

But:

$$n \notin \Theta(n^2), \quad n \notin \Theta(\log n)$$

because the growth rates are not the same.

Key Idea:
Big-O gives an upper limit, Big-Ω gives a lower limit, Big-Θ gives the exact growth rate.
A function can have many upper bounds and many lower bounds, but only one tight asymptotic family.

3 Coming next

3.1 Guaranteed exam marks